ResumeCraft

A Component-Driven, Client-Side Resume Builder Built with Vue 3

Vue Project Report

Submitted by Irha Shehzadi (036) & Zamna AllahYar (071)

Department of Software Engineering

July 2026

Abstract

Almost everyone who has ever applied for a job has faced the same small but irritating problem: getting a resume to look right. Word documents drift out of alignment the moment a bullet point runs one line too long, fonts change unexpectedly when the file is opened on a different computer, and formatting a second version for a different job application usually means copying the whole document and hoping nothing breaks. ResumeCraft was built to remove that friction. It is a single-page web application, written entirely in Vue 3, that lets a user fill in one clean form and watch a fully formatted resume take shape beside it in real time.

This report documents the complete design and implementation of ResumeCraft. It explains why the application is organised into three routed views instead of one long page, why a single shared composable was chosen to hold the resume data instead of a heavier state-management library, how the Props-down and Emits-up pattern keeps every form component predictable, and how Local Storage is used to make sure a user's work survives a page refresh without needing a server or a database. Every claim made in this report is grounded directly in the ResumeCraft source code — nothing here describes a feature, a library, or a technology that is not actually present in the project.

By the end of the report, the reader should be able to explain, without referring back to the code, how a keystroke typed into the Personal Information form eventually reaches Local Storage, and how that same piece of data reappears instantly on the Preview page. That single journey — from an input field, through an emitted event, into a reactive object, and finally onto the screen and into the browser's storage — is the thread that ties the entire application together, and it is the thread this report follows from start to finish.

Keywords: Vue 3, Composition API, Vue Router, reactive state, Props and Emits, composables, Local Storage, single-page application, client-side architecture.

Table of Contents

1. Introduction

Every software project begins with a small, specific irritation that the author decided was worth solving. For ResumeCraft, that irritation was the resume-writing process itself. Traditional word processors are built for general documents, not for the very particular, very structured shape a resume actually has — a name and a title at the top, a short summary, a block of work experience with consistent dates, a block of education, and a list of skills. Nothing about that structure needs to be reinvented every time someone applies for a job, and yet most people rebuild it from scratch in a blank document every single time.

ResumeCraft takes a different approach. Instead of a blank page, it gives the user a form that already understands the shape of a resume. Instead of manual formatting, it renders a live, styled preview the instant a field is edited. And instead of asking the user to create an account or install anything, it runs entirely inside the browser and remembers the user's data using Local Storage.

1.1 Motivation

The guiding idea behind the project was simple: separate the job of collecting information from the job of presenting it. A user should never have to think about spacing, alignment, or font size while they are trying to describe their last job. That responsibility belongs entirely to the application. This single idea is the reason the project is split so cleanly into form components on one side and a single preview component on the other — a decision that shows up again and again throughout this report.

1.2 Objectives

- Allow a user to enter personal information, work experience, education, and skills through a guided form.
- Reflect every change instantly in a formatted, resume-shaped preview, without requiring the user to click a 'refresh' or 'update' button.
- Preserve the user's data across page reloads and across the different pages of the application, without requiring a backend server or a database.
- Allow the finished resume to be exported as a PDF using nothing more than the browser's own printing capability.
- Keep the codebase organised so that each component has exactly one clearly defined responsibility, making the application easy to extend later.

1.3 Scope of the Report

This report walks through ResumeCraft in the same order that a new developer joining the project would want to explore it: first the overall shape of the application, then how the user moves between its pages, then how the individual building blocks are organised, then how those building blocks share and update data, and finally how that data is preserved between visits. Each chapter builds directly on the one before

it, because that is also how the application itself was built — routing had to exist before views could be separated, views had to exist before components could be assembned inside them, and components had to exist before a shared state layer was needed to keep them synchronised.

2. Project Overview

Before looking at any code, it helps to see the application the way a user sees it. ResumeCraft presents three pages, reachable through a persistent navigation bar at the top of the screen: a Home page that introduces the product, a Builder page where the resume is actually written, and a Preview page where the finished document is displayed and printed.

2.1 What the Application Does

On the Home page, the user is introduced to the idea of the tool and given two buttons — one that leads straight into the Builder, and one that jumps directly to the Preview in case a resume has already been started. The Builder page is where the real work happens. It lays out four forms in sequence — Personal Information, Work Experience, Education, and Skills — with a live preview positioned beside them that updates on every keystroke. The Preview page removes the forms entirely and shows only the finished resume, along with an Export as PDF button that triggers the browser's native print dialogue.

2.2 Feature Summary

Feature	Description
Guided data entry	Separate forms for personal details, experience, education, and skills, each with clearly labelled fields and placeholder examples.
Live preview	A resume-shaped preview updates immediately as the user types, without any explicit save or refresh action.
Multiple entries	Experience, education, and skills each support any number of entries, added or removed independently.
Data persistence	The complete resume is written to the browser's Local Storage after every change, so it survives a page reload.
Shared state across pages	The Builder and Preview pages display exactly the same data because they read from the same shared object.
PDF export	The Preview page includes an Export as PDF action built on the browser's own window.print() function.
Responsive layout	The layout adapts through CSS media queries so the Builder and Preview remain usable on narrower screens.

Table 1. Summary of ResumeCraft's core features.

2.3 Target Users

The application is aimed at anyone who needs to produce a clean, conventional resume without wrestling with a word processor — students preparing their first CV, professionals updating their experience for a new application, or anyone who simply wants a faster way to keep a resume current. Because there is no

login and no server-side account, the tool is meant to be opened, used, and closed quickly, with the browser itself quietly remembering the user's information for next time.

3. System Architecture

With a sense of what the application does, the next question is how it is put together. ResumeCraft is a purely client-side application — there is no backend server, no database, and no network request anywhere in the codebase. Everything the user sees and everything the user's data touches happens inside the browser. That single fact shapes almost every architectural decision described in this report.

3.1 High-Level Architecture

The application is built with Vue 3, using the Composition API and the <script setup> syntax throughout every component. Navigation between pages is handled by Vue Router. The data the user enters is held in one shared reactive object, created inside a composable function, and that same object is read by every page and every component that needs it. Persistence is handled by the browser's Local Storage API, wrapped inside that same composable so that no component has to talk to Local Storage directly.

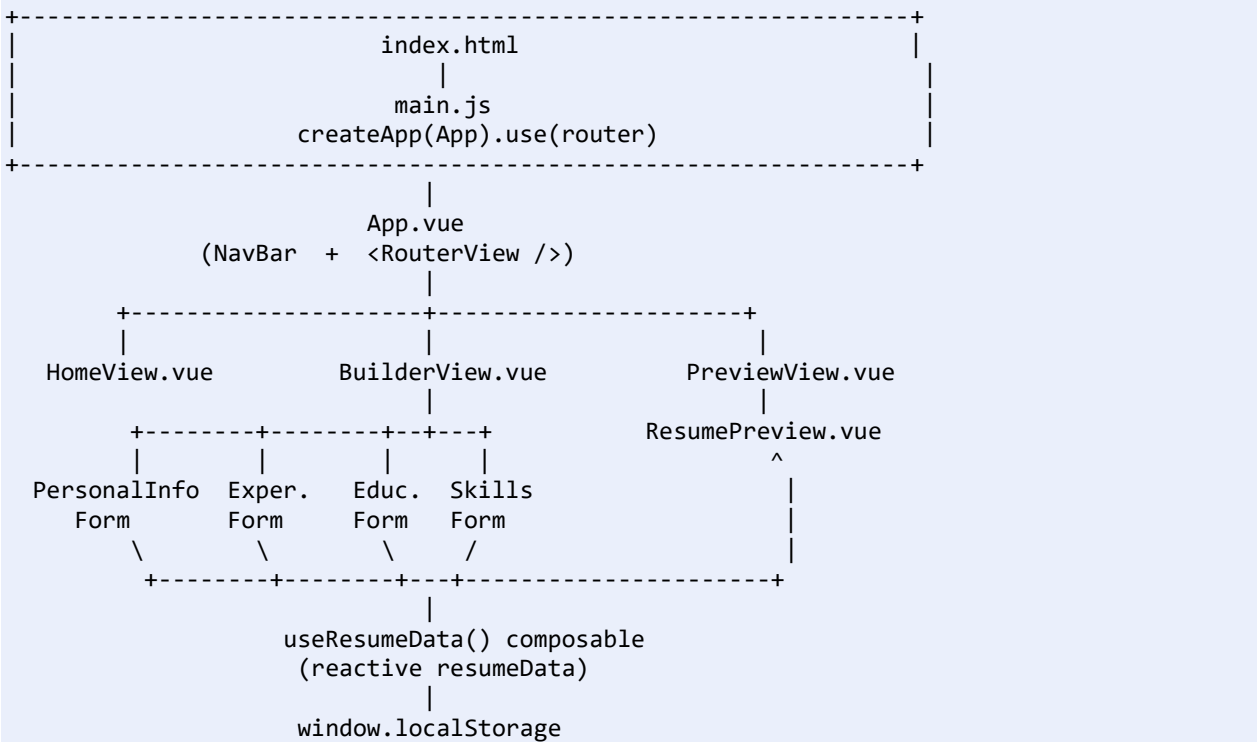

Figure 1. Overall application architecture, from the HTML entry point down to the browser's Local Storage.

Figure 1 shows the whole application in one picture, and it is worth reading from top to bottom before going any further. At the very top, index.html loads main.js, which is the one line of code that actually starts the Vue application. Below that sits App.vue, the shell that every page lives inside. Below App.vue, Vue Router decides which of the three views to display. Inside BuilderView, four separate form components are assembled side-by-side with the preview component. And underneath all of that, tying

everything together, is the useResumeData composable — the single object that every one of these components ultimately reads from and writes to.

3.2 Why No Backend?

A resume builder does not need a server to do its job. All the logic required — collecting text, formatting it, and letting the user print it — can happen entirely on the user's own machine. Introducing a backend would have added a database, an API layer, and user accounts, none of which the project actually needs. Choosing to keep everything client-side is therefore not a limitation of the project; it is a deliberate simplification that matches the size of the problem being solved.

3.3 Why Vue 3 and the Composition API?

Every component in ResumeCraft is written with <script setup>, the shorthand form of Vue 3's Composition API. The reason this matters becomes clear the moment two components need to share data — which, in this project, happens constantly. The Composition API lets logic like resumeData and its update functions live inside a plain JavaScript function (useResumeData) instead of being scattered across each component's individual options. That one function can then be imported anywhere it is needed, which is exactly the mechanism the next chapters rely on.

4. Folder Structure

Before exploring how the pieces of ResumeCraft talk to each other, it helps to see where each piece actually lives. The project follows the conventional layout produced by the standard Vue command-line tooling, with one addition: a composables folder, created specifically to hold the shared state logic described later in this report.

```
src/
├── main.js                 Application entry point
├── App.vue                 Root component: NavBar + RouterView
├── router/
│   └── index.js            Route definitions (Home, Builder, Preview)
├── views/
│   ├── HomeView.vue        Landing page
│   ├── BuilderView.vue     Form-entry workspace with live preview
│   └── PreviewView.vue     Full-page resume preview + PDF export
├── components/
│   ├── NavBar.vue          Top navigation bar
│   ├── PersonalInfoForm.vue Personal details form
│   ├── ExperienceForm.vue  Work experience list form
│   ├── EducationForm.vue   Education list form
│   ├── SkillsForm.vue      Skills list form
│   └── ResumePreview.vue   Read-only rendered resume
├── composables/
│   └── useResumeData.js    Shared reactive state + Local Storage logic
└── styles/
    └── main.css            Design tokens and shared utility classes
```

Figure 2. Project folder structure.

4.1 Purpose of Each Folder

Folder / File	Responsibility
router/	Defines which URL path maps to which view, and nothing else.
views/	Page-level components, each mapped to exactly one route.
components/	Reusable, single-purpose building blocks used inside the views.
composables/	Framework-agnostic logic that manages shared state, kept separate from the UI layer.
styles/	Global design tokens (colours, spacing, fonts) and utility classes shared by every component.

Table 2. Responsibility of each top-level folder.

This separation is not just tidiness for its own sake. Keeping composables separate from components means the resume data logic could, in principle, be reused by a completely different UI — a mobile app, a command-line tool, anything — without changing a single line of useResumeData.js. That is the real

benefit of pulling state logic out of the components and into its own file, and it is a theme this report returns to in Chapter 7.

5. Routing Architecture

Once the folder structure is clear, the next natural question is how a user actually moves between the Home, Builder, and Preview pages. A traditional website would reload the entire page every time the user clicked a link, discarding everything already loaded and fetching a brand-new HTML document from a server. ResumeCraft avoids that entirely by using Vue Router, which lets the visible view change instantly while the rest of the page — the navigation bar, the loaded JavaScript, the application's memory — stays exactly where it is.

5.1 Why Client-Side Routing Matters Here

This matters more for ResumeCraft than it would for a simple content website, because the whole point of the application is that the user's resume data has to survive a page change. If clicking 'Preview' triggered a full page reload, the browser would tear down the entire JavaScript application — including the reactive resumeData object — and the user's work would vanish. Vue Router avoids this by intercepting the click, updating the URL in the address bar, and swapping out only the component displayed inside <RouterView>, without ever asking the browser to fetch a new page.

5.2 Route Configuration

The route table lives in router/index.js, created with Vue Router's createRouter() function. It uses createWebHistory(), which keeps the URLs clean (for example /builder rather than /#/builder) by relying on the browser's History API.

```
const router = createRouter({
  history: createWebHistory(),
  routes: [
    { path: '/', name: 'home', component: HomeView },
    { path: '/builder', name: 'builder', component: BuilderView },
    { path: '/preview', name: 'preview', component: PreviewView }
  ]
})
```

Listing 1. Route configuration from router/index.js.

Path	Name	Component	Purpose
/	home	HomeView.vue	Introduces the application and links to the other two pages.
/builder	builder	BuilderView.vue	Where the user fills in every section of the resume.
/preview	preview	PreviewView.vue	Shows the finished resume and offers PDF export.

Table 3. The three routes defined in ResumeCraft.

Three routes is a deliberately small number, and that smallness is itself a design decision. ResumeCraft was never meant to have separate pages for each resume section; splitting Personal Information, Experience, Education, and Skills into four different routes would force the user to click back and forth constantly while filling in a single document that is naturally read as one continuous whole. Keeping the entire editing experience on one Builder route, with the four forms stacked together, matches how a resume is actually written — all its sections at once, referring back and forth between them.

5.3 RouterView and RouterLink

App.vue contains a single <RouterView /> element. This is the placeholder Vue Router uses to display whichever component matches the current route; it never appears twice, and it never needs to know which specific view it currently contains. Navigation itself happens through <RouterLink> elements, visible both in NavBar.vue (Home / Builder / Preview) and inside individual views, such as the 'Go to full preview' button at the bottom of BuilderView. A RouterLink behaves like a normal anchor tag from the user's point of view, but instead of asking the browser to load a new page, it asks Vue Router to switch the component shown inside RouterView.

With navigation explained, the next question becomes far more interesting: once the user is inside the Builder page and starts typing, where does that information actually go, and how does it reach the Preview page a moment later? Answering that question requires looking at how ResumeCraft's components are organised, and then at the shared state layer sitting underneath all of them.

6. Component Architecture

A resume has several distinct sections, and each one asks for slightly different information — a school and a degree are not the same thing as a company and a job title. Rather than build one enormous form component that tries to handle every field for every section, ResumeCraft splits the interface into small, single-purpose components, each responsible for exactly one part of the resume.

6.1 Component Responsibility Table

Component	Responsibility
NavBar.vue	Displays the top navigation links. Holds no resume data at all.
PersonalInfoForm.vue	Collects name, job title, contact details, and the summary paragraph.
ExperienceForm.vue	Collects one or more work experience entries.
EducationForm.vue	Collects one or more education entries.
SkillsForm.vue	Collects one or more skill entries, each with a name and proficiency level.
ResumePreview.vue	Renders the complete resume. Read-only; never changes any data.
BuilderView.vue	Assembles all four forms and the preview, and connects them to the shared state.
PreviewView.vue	Displays ResumePreview full-size, with the Export as PDF control.

Table 4. Responsibility of every Vue component in the project.

Every one of the four form components — PersonalInfoForm, ExperienceForm, EducationForm, and SkillsForm — follows the exact same internal pattern, which is worth naming explicitly because it appears throughout the codebase: Props flow down into the component carrying the current data, and Emits flow back up carrying the requested change. No form component ever changes the shared resumeData object directly. This is deliberate, and the next section explains exactly why.

6.2 Props-Down, Emits-Up

In Vue, a Prop is a piece of data that a parent component passes down into a child component. It is read-only from the child's point of view — the child can display it, but it is not supposed to change it directly. An Emit is the opposite direction: it is a custom event that a child component sends upward to its parent, usually carrying some new value along with it. Together, Props and Emits give Vue components a clear, one-directional communication channel: data flows down, requests to change that data flow up.

ResumeCraft leans on this pattern everywhere. Take PersonalInfoForm.vue as an example. It receives the current personalInfo object as a Prop, and every input field reads its displayed value straight from that

Prop rather than from any local variable of its own. When the user types into the 'Full name' field, the component does not update personalInfo itself. Instead, it calls handleFieldChange(), which builds a brand-new object — the old personalInfo spread out, with just the one changed field overwritten — and emits it upward through an event simply named update.

```
function handleFieldChange(field, value) {
  emit('update', { ...props.personalInfo, [field]: value })
}
```

Listing 2. handleFieldChange() inside PersonalInfoForm.vue.

The three list-style forms — ExperienceForm, EducationForm, and SkillsForm — extend this same idea to handle a whole array of entries instead of a single object. Each of them emits three different events: add-entry when the user clicks the 'Add' button, update-entry (carrying the specific entry's id together with its new values) whenever a field inside one entry changes, and remove-entry when the user removes a specific entry.

Event	Emitted By	Payload	Meaning
update	PersonalInfoForm	Updated personalInfo object	One field in the personal details changed.
add-entry	Experience / Education / Skills forms	None	The user wants a new blank entry created.
update-entry	Experience / Education / Skills forms	Entry id, updated entry object	One field inside a specific entry changed.
remove-entry	Experience / Education / Skills forms	Entry id	The user wants a specific entry deleted.

Table 5. Every custom event emitted by the form components.

6.3 Why Not Let Each Form Hold Its Own Data?

It would certainly be possible to let each form component keep a private copy of its section's data and only report a finished value once the user moves away. ResumeCraft deliberately avoids that design, because the live preview would then be showing stale information until the user left the field — defeating the entire purpose of a preview that updates as you type. By having every keystroke emit an update immediately, and by having the single shared resumeData object be the only place that data is actually stored, the preview can simply watch that one object and always be correct, without any component needing to specifically notify it.

6.4 ResumePreview: A Deliberately Reusable Component

ResumePreview.vue receives one single Prop — the entire resume object — and renders it. It defines no Emits at all, because it has nothing to ask its parent to change; its only job is to display data faithfully. This

component is used in two very different places: inside BuilderView, scaled down and pinned to the side of the screen as a live preview, and inside PreviewView, shown at full size as the final, printable document.

Reusing the exact same component in both places is not a minor convenience. It guarantees, by construction, that the live preview the user sees while editing and the final resume they print are always identical, because they are rendered by the same template with the same logic. If the project instead maintained two separate preview implementations, every future change to the resume's layout would have to be made twice, and the two versions would eventually drift out of sync. Reusability here is really a correctness guarantee, not just a way to save typing.

With the individual components now understood, the natural next question is where the data they display actually comes from in the first place, and how the same resumeData object manages to exist in more than one component at the same time.

7. State Management

A resume builder is only useful if the Builder page and the Preview page agree on what the resume currently contains. If each page kept its own private copy of the data, editing an entry on the Builder page would have no effect on what the Preview page displayed, because the two pages would simply be looking at two different objects. ResumeCraft solves this with a single shared reactive object, created once inside a composable function called useResumeData.

7.1 What Is a Composable?

In Vue's Composition API, a composable is simply a regular JavaScript function whose name conventionally starts with 'use', which packages together some reactive state and the logic needed to manage it. Any component can call that function to gain access to the same state and the same logic. It is Vue's answer to a common problem: how do you share behaviour between components without resorting to a global variable that bypasses Vue's reactivity system, and without introducing a full state-management library.

7.2 Why reactive() and Not ref()

Vue's reactive() function takes a plain JavaScript object and returns a version of it that Vue can watch for changes. Whenever a property inside that object changes, every part of the interface that depends on that property re-renders automatically. ResumeCraft uses reactive() specifically because resumeData is naturally a single structured object — personal details, an experience list, an education list, and a skills list — and reactive() is the more natural fit for an object with this shape, compared to ref(), which is more commonly reached for when wrapping a single, standalone value.

```
const resumeData = reactive({
  personal: {
    fullName: '', jobTitle: '', email: '',
    phone: '', location: '', summary: ''
  },
  education: [],
  experience: [],
  skills: []
})
```

Listing 3. The reactive resumeData object at the heart of useResumeData.js.

7.3 Why Module Scope Makes This Object Shared

The most important detail in useResumeData.js is easy to miss on a first read: resumeData is declared at the top level of the file, outside of the useResumeData() function itself. Because JavaScript's module system caches a module the first time it is imported, every single component that writes import { useResumeData } from '...' is importing the exact same file, and therefore receiving a reference to the exact same resumeData object — not a fresh copy. This is the entire mechanism that lets BuilderView and

PreviewView, two completely separate route components, display identical data without ever being told about each other.

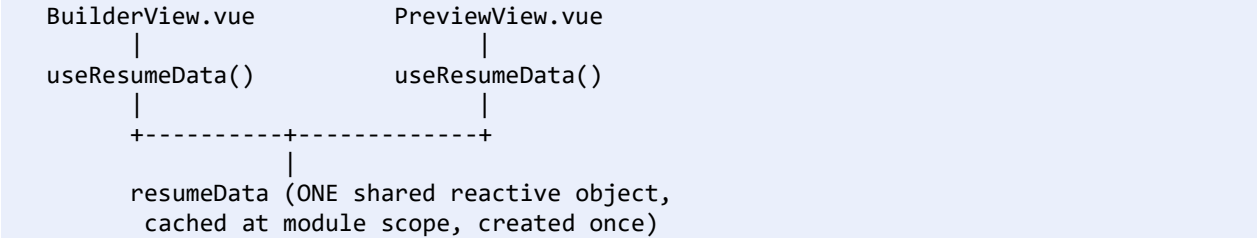

Figure 3. Both views import the same module-scoped reactive object.

7.4 Why Not Pinia or Vuex?

Pinia and Vuex are both dedicated state-management libraries commonly used in larger Vue applications, designed to manage many interconnected pieces of state across dozens of components, often with features like time-travel debugging, modules, and plugins. ResumeCraft has exactly one piece of shared state — the resume itself — read by a small, fixed set of components. Reaching for a full state-management library to manage a single object would add a dependency and a layer of configuration the project does not need. A hand-written composable achieves precisely the same sharing behaviour with far less code, and with nothing hidden behind a library's internal machinery. This mirrors the same reasoning that ruled out a backend in Chapter 3: the solution should match the size of the actual problem, not the size of problems the project does not have.

7.5 The Update Functions

useResumeData.js does not just expose the raw resumeData object — it also exposes a specific set of functions for changing it: updatePersonalInfo, addEducation, updateEducation, removeEducation, addExperience, updateExperience, removeExperience, addSkill, updateSkill, and removeSkill. Every one of these functions follows the same shape: locate the right piece of resumeData, replace or modify it immutably, and then call persist() to save the updated object to Local Storage.

```
function updateEducation(id, updatedEntry) {
  const index = resumeData.education.findIndex(item => item.id === id)
  if (index !== -1) {
    resumeData.education[index] = { ...updatedEntry, id }
    persist()
  }
}
```

Listing 4. updateEducation(), showing the find-modify-persist pattern repeated across the file.

Centralising every mutation inside these named functions means only one place in the entire codebase is allowed to change resumeData — the composable itself. No component ever reaches into resumeData.education and edits it directly; it always goes through addEducation, updateEducation, or

removeEducation. This keeps the rule 'only the composable writes state' completely unambiguous, which becomes especially valuable as the project grows and more developers might touch the code.

Every one of these functions relies on being able to identify one entry inside a list of many, which raises a small but important question: where do these ids actually come from? That detail, along with how the composable restores previously saved data when the page first loads, is where the discussion turns next.

7.6 Generating Unique Ids

Because education, experience, and skills are all stored as arrays, and any entry within them might be added or removed independently, each entry needs a stable identifier so that Vue's :key binding and the update/remove functions can reliably target the correct one. useResumeData.js keeps a simple module-scoped counter, nextId, and a small generateId() function that returns the current value and increments it.

```
let nextId = 1
function generateId() {
  return nextId++
}
```

Listing 5. The id-generation counter used for every new entry.

This is a deliberately lightweight approach — no external library, no UUIDs — because within a single browser session, a simple incrementing counter is entirely sufficient to guarantee that no two entries ever collide.

8. Data Flow: From Keystroke to Screen

The previous two chapters explained the two halves of the puzzle separately: components communicate through Props and Emits, and a shared composable holds the actual data. This chapter puts both halves together and follows one single keystroke all the way through the application, because seeing the full journey in one place makes the whole architecture click into place.

8.1 The Complete Journey of One Keystroke

Imagine the user is on the Builder page and types a letter into the 'Full name' field.

```
1. User types into <input> inside PersonalInfoForm.vue
        |
        v  (@input event)
2. handleFieldChange('fullName', value) builds a new object
   { ...personalInfo, fullName: value }
        |
        v  emit('update', newObject)
3. BuilderView.vue is listening: @update="updatePersonalInfo"
        |
        v
4. updatePersonalInfo() inside useResumeData.js runs:
     resumeData.personal = { ...updatedPersonal }
     persist()   -->  localStorage.setItem(...)
        |
        v  (Vue's reactivity system notices the change)
5. ResumePreview.vue re-renders automatically,
   because it reads resume.personal.fullName directly
        |
        v
6. The updated name appears instantly in the live preview
```

Figure 4. The complete lifecycle of a single keystroke, from input field to on-screen preview and Local Storage.

Nothing in this chain has to be manually wired for a specific field. Because ResumePreview simply reads resume.personal.fullName inside its template, and because resume.personal is a reactive object, Vue's own dependency-tracking system automatically knows to re-render that part of the preview the moment the property changes. No component ever had to say 'and now update the preview' explicitly — that responsibility is handled entirely by Vue's reactivity system, once the underlying data was made reactive in the first place.

8.2 Why BuilderView Is the Coordinator

BuilderView.vue is the one place in the application that both calls useResumeData() and passes the resulting data and functions down into every form. It is deliberately positioned as a coordinator rather than a component with logic of its own: it does not know how any form collects its data, and it does not know how ResumePreview renders it. Its entire job is to connect the pieces — pass resumeData.personal into PersonalInfoForm, and forward the resulting update event straight into updatePersonalInfo. This

keeps BuilderView small and easy to read, and keeps every form fully independent of how the rest of the application happens to be wired together.

```
<PersonalInfoForm
  :personal-info="resumeData.personal"
  @update="updatePersonalInfo"
/>
```

Listing 6. BuilderView.vue simply forwards Props down and Emits up — it never touches resumeData directly.

8.3 Why the Preview Page Never Writes Data

PreviewView.vue calls useResumeData() purely to read resumeData, and passes it straight into ResumePreview as a Prop. It defines no update functions of its own, and ResumePreview itself defines no Emits. This is intentional: allowing only the Builder page and its forms to ever change the resume keeps the rule 'who is allowed to write state' completely unambiguous. If the Preview page were ever allowed to modify data too, tracking down where an unexpected change came from would become far harder as the application grew.

9. Local Storage and Data Persistence

Reactivity alone solves the problem of keeping the Builder and Preview pages synchronised while the application is open, but it does nothing to help a user who accidentally refreshes the browser, or who closes the tab intending to come back tomorrow. Without some form of persistence, resumeData would simply be recreated empty every time the page loads, because it lives only in the browser's memory. ResumeCraft solves this with the Web Storage API's Local Storage, which lets a website save small amounts of text data directly in the user's browser, associated with that specific website, surviving reloads and browser restarts alike.

9.1 Saving Data: persist()

Every single update function inside useResumeData.js ends with a call to persist(), a small helper that converts the entire resumeData object to a JSON string and writes it under one fixed key.

```
const STORAGE_KEY = 'resumecraft-data'

function persist() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(resumeData))
}
```

Listing 7. persist(), called after every mutation to resumeData.

Saving the entire object on every change, rather than trying to patch individual keys inside Local Storage, keeps the persistence logic extremely simple: there is exactly one thing being saved, under exactly one key, and it is always a complete, self-consistent snapshot of the resume.

9.2 Restoring Data: loadResumeData()

The opposite operation happens in loadResumeData(), which reads that same key back out of Local Storage, parses the JSON string, and copies the result into the fields of resumeData.

```
function loadResumeData() {
  const saved = localStorage.getItem(STORAGE_KEY)
  if (!saved) return
  try {
    const parsed = JSON.parse(saved)
    resumeData.personal = { ...resumeData.personal, ...parsed.personal }
    resumeData.education = parsed.education || []
    resumeData.experience = parsed.experience || []
    resumeData.skills = parsed.skills || []
    // ...also restores nextId, described below
  } catch (error) {
    console.error('Could not read saved resume data:', error)
  }
}
```

Listing 8. loadResumeData(), showing the try/catch guard around JSON parsing.

Wrapping the parsing step inside a try/catch block matters because Local Storage simply stores text — nothing stops that text from being corrupted or manually edited by the user through their browser's developer tools. Should JSON.parse() ever fail, the catch block logs the problem instead of crashing the entire application, and resumeData simply keeps whatever default values it already had.

9.3 Keeping Generated Ids Ahead of Restored Data

When resumeData is restored from a previous session, its education, experience, and skills entries already carry ids that were generated during that earlier session. If nextId were left at its default starting value of 1, any new entry created afterwards could collide with an id that already exists in the restored data. loadResumeData() therefore collects every id already present across all three lists and sets nextId to one more than the largest id found, guaranteeing every future id remains unique.

```
const allIds = [
  ...resumeData.education.map(item => item.id),
  ...resumeData.experience.map(item => item.id),
  ...resumeData.skills.map(item => item.id)
]
if (allIds.length) {
  nextId = Math.max(...allIds) + 1
}
```

Listing 9. Restoring nextId so that generated ids never collide with previously saved ones.

9.4 Why loadResumeData() Runs at Module Level

At the very bottom of useResumeData.js, outside of any function, sits a single line: loadResumeData(). Because this line executes the moment the module is first imported, rather than waiting for a component's lifecycle hook, the saved resume is already restored before the very first component even finishes being created — no matter which of the three pages the user happens to land on first. This is why App.vue's own comment notes that no onMounted() hook is needed there for this purpose: the timing guarantee comes from where the JavaScript module system runs the code, not from any particular component's lifecycle.

9.5 Local Storage Schema

Key	Value Shape	Written By	Read By
resumecraft-data	JSON string of the full resumeData object (personal, education, experience, skills)	persist(), called after every mutation	loadResumeData(), called once at module import

Table 6. The single Local Storage key used by the entire application.

Persistence solved the problem of remembering data between visits. The one remaining question is what happens once the resume is finished and the user actually wants to walk away with something they can attach to a job application — and that is where PDF export comes in.

10. User Interface Design

Good architecture on its own does not make an application pleasant to use. ResumeCraft's visual design was built around a single, deliberately restrained set of design tokens, defined once at the top of styles/main.css and reused everywhere, rather than scattering colours and spacing values throughout individual components.

10.1 Design Tokens

The stylesheet defines its palette, typography, spacing scale, radii, and shadows as CSS custom properties on the :root selector. A soft ivory background, a deep navy ink colour for text, and a single restrained royal-blue accent form the entire colour palette — a choice made specifically so the interface would read the way a well-typeset resume reads, rather than looking like a typical, brightly coloured web app.

Token Category	Examples	Purpose
Colour	--color-bg, --color-ink, --color-accent	One consistent palette reused across every component.
Typography	--font-display (Fraunces), --font-body (Inter)	A serif display font for headings, a clean sans-serif for body text.
Spacing	--space-1 through --space-8	A fixed spacing scale so padding and margins stay visually consistent.
Radius & Shadow	--radius-sm/md/lg, --shadow-sm/md/lg	Consistent rounded corners and elevation across cards and buttons.

Table 7. Categories of design tokens defined in main.css.

10.2 Shared Utility Classes

Rather than repeating the same button or card styling inside every component's <style> block, main.css defines a small set of shared classes — .card, .btn, .field-group, .field-row — that every component then reuses. This is the same underlying philosophy as the useResumeData composable applied to styling instead of state: define a shared piece of behaviour once, in one place, and let every component that needs it simply reference it, instead of quietly re-implementing a slightly different version of the same thing five separate times.

10.3 Responsive Layout

Both the Builder page's two-column layout (forms beside the live preview) and the resume card itself collapse into a single column on narrower screens, controlled through @media (max-width: ...) queries placed inside each component's own scoped styles. This keeps the Builder usable on a tablet or a narrow browser window without requiring an entirely separate mobile layout.

10.4 Accessibility Considerations

main.css includes a visible focus style, applied through the :focus-visible pseudo-class, on every interactive element — links, buttons, inputs, textareas, and selects. This ensures that a user navigating the application with a keyboard rather than a mouse can always see exactly which element is currently focused. The stylesheet also respects the prefers-reduced-motion media feature, disabling animations and transitions for users who have requested reduced motion at the operating-system level.

11. Complete User Workflow

Having examined the application piece by piece, it is worth stepping back and tracing the entire journey a user takes, from the very first visit to walking away with a finished PDF.

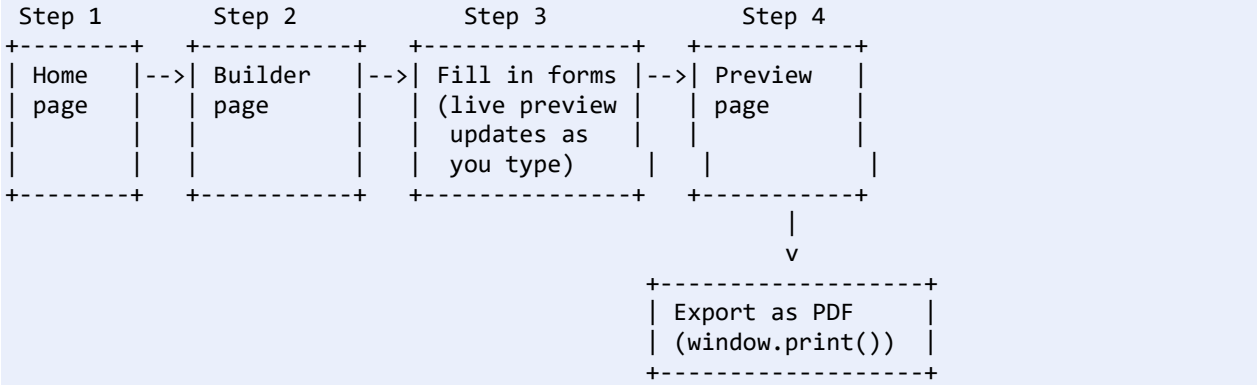

Figure 5. The complete end-to-end user workflow, from landing on the Home page to exporting a PDF.

11.1 Step-by-Step Walkthrough

- The user opens ResumeCraft and lands on the Home page, which introduces the tool and offers two entry points: 'Start building' and 'View my resume'.
- Choosing 'Start building' navigates to the Builder page, where useResumeData() is called and the (possibly empty, possibly previously saved) resumeData object is loaded into view.
- The user fills in the Personal Information form; each keystroke flows through the update event described in Chapter 8 and appears instantly in the live preview beside the form.
- The user adds one or more work experience entries using the '+ Add experience' button, filling in company, role, dates, and a description for each.
- The same process repeats for Education and Skills, each supporting any number of entries.
- At any point, the user can refresh the browser or close the tab; because every change was already persisted through persist(), the next time useResumeData() runs, loadResumeData() restores everything exactly as it was left.
- Once satisfied, the user navigates to the Preview page, which displays the exact same resumeData object at full size, styled as a finished, printable document.
- Clicking 'Export as PDF' calls window.print(), and the browser's own print styles (defined under the @media print rule in main.css) hide everything except the resume itself, letting the user save the result as a PDF through their browser's print dialogue.

11.2 Handling an Empty Resume

Both the individual form components and ResumePreview itself account for the case where no data has been entered yet. Each list-based form (Experience, Education, Skills) displays a short empty-state message — for example, 'No experience added yet. Click Add experience to create your first entry.' — instead of showing a blank list with no explanation. ResumePreview goes a step further: if there is no

name, no experience, no education, and no skills at all, it replaces the entire resume layout with a single friendly message directing the user back to the Builder.

12. PDF Export and the Print Workflow

A resume that only exists inside a browser tab is not very useful; at some point it has to become a file the user can actually attach to an email or upload to a job portal. Rather than generate a PDF on a server, or bundle a JavaScript PDF-generation library into the project, ResumeCraft relies entirely on a feature every browser already has: printing.

12.1 window.print()

PreviewView.vue defines a small handlePrint() function that does nothing more than call the browser's built-in window.print() method, wired to the 'Export as PDF' button's click event.

```
function handlePrint() {
  window.print()
}
```

Listing 10. handlePrint() in PreviewView.vue.

Calling window.print() opens the browser's native print dialogue, where the user can choose 'Save as PDF' as the destination instead of a physical printer. This avoids adding any PDF-generation dependency to the project entirely, at the small cost of relying on the browser's own print rendering rather than a custom-built one.

12.2 Print-Specific Styling

Simply calling window.print() would, by default, print the entire page exactly as it appears on screen — including the navigation bar and the 'Export as PDF' button itself, neither of which belong in a printed resume. main.css solves this with a dedicated @media print block that hides every element on the page and then selectively makes only the resume itself, identified by the id resume-print-area, visible again.

```
@media print {
  body * { visibility: hidden; }
  #resume-print-area,
  #resume-print-area * { visibility: visible; }
  #resume-print-area {
    position: absolute;
    top: 0; left: 0; width: 100%;
    margin: 0; box-shadow: none; border: none;
  }
  @page { margin: 12mm; }
}
```

Listing 11. The @media print rule from main.css, isolating the resume for printing.

The id resume-print-area is placed on the root <article> element inside ResumePreview.vue. Because this same component is reused on both the Builder page and the Preview page, the print rule technically applies wherever ResumePreview is rendered — but in practice, the export action is only exposed to the

user from the Preview page, which is exactly where a clean, full-size version of the resume is already being displayed.

13. Testing and Verification

Verifying an interactive, form-heavy application like ResumeCraft is less about checking a single calculation and more about confirming that data keeps flowing correctly between components as the user interacts with the interface. Testing was carried out manually, working systematically through each feature described in the earlier chapters.

13.1 Manual Test Checklist

#	Test Case	Expected Result
1	Type into any Personal Information field	The live preview updates immediately with the new value.
2	Click '+ Add experience' / '+ Add education' / '+ Add skill'	A new blank entry appears in the form, and the empty-state message disappears.
3	Fill in a new experience entry	The entry appears correctly formatted in the live preview, including the date range.
4	Click 'Remove' on a specific entry	Only that entry disappears; all other entries remain unaffected.
5	Refresh the browser after entering data	All previously entered data reappears exactly as it was left.
6	Navigate from Builder to Preview	The Preview page shows exactly the same data as the Builder's live preview.
7	Click 'Export as PDF' on the Preview page	The browser's print dialogue opens, showing only the resume content.
8	Leave every field empty and open the Preview page	The 'Nothing to preview yet' message is displayed instead of an empty layout.
9	Resize the browser window below 640px	The Builder and Preview layouts collapse into a single column.
10	Tab through the form using only the keyboard	A visible focus outline appears on every interactive element in sequence.

Table 8. Manual test cases used to verify ResumeCraft's behaviour.

13.2 Why Manual Testing Was Appropriate Here

ResumeCraft contains very little independent business logic to unit test in isolation — most of its behaviour is the direct, visible result of Vue's reactivity system responding to Props, Emits, and changes to resumeData. The most meaningful way to verify this kind of application is to interact with it the same way a real user would and confirm that the interface responds correctly at every step, which is exactly what the checklist above does.

14. Future Improvements

ResumeCraft, as it stands, fully satisfies the objectives laid out in Chapter 1. There are, however, a number of directions the project could reasonably grow into, each following naturally from a design decision already made.

- Multiple resume templates: because ResumePreview is already the single, isolated component responsible for rendering, adding a second visual template would mean creating an alternative version of this one component, without touching the composable or the forms at all.
- Reordering entries: drag-and-drop reordering of experience or education entries could be added by extending the existing array-based structure inside resumeData, without changing the overall Props/Emits pattern.
- Export to additional formats: alongside the existing print-to-PDF workflow, direct export to formats like .docx could be introduced as an additional action on the Preview page.
- Cloud backup: for a user who wants their resume available on more than one device, the existing persist()/loadResumeData() pair could be extended to optionally sync with a remote account, while keeping Local Storage as the default, no-login experience.
- Section-level validation: gentle warnings (for example, a job title left blank) could be layered on top of the existing forms without changing how data already flows through the application.

Every one of these possible extensions builds on top of the existing architecture rather than requiring it to be rebuilt, which is, in itself, a small piece of evidence that the underlying design decisions described throughout this report were the right ones for a project of this size.

15. Conclusion

ResumeCraft set out to solve a small, specific, and very familiar problem: making it easy to write a well-formatted resume without wrestling with a general-purpose word processor. Over the course of this report, that problem has been traced all the way down to its implementation — from the three routes that structure the application, through the Props-down/Emits-up pattern that keeps every form predictable, into the single shared reactive object that lets the Builder and Preview pages agree on what the resume actually contains, and finally out to Local Storage, where that same data quietly survives every page refresh.

What ties all of these pieces together is a single, consistent architectural instinct: keep responsibilities separate, and keep the solution no larger than the problem actually requires. Forms collect data and never store it. The composable stores data and never renders anything. The preview renders data and never changes it. No backend was introduced because none was needed; no external state-management library was introduced because a single composable did the job just as well, with far less complexity. Every one of these choices, examined individually in the chapters above, adds up to an application that is small, but genuinely well organised — exactly the kind of foundation a growing project needs.

Building ResumeCraft, and then documenting it carefully enough to explain in this report, made the underlying ideas behind Vue's Composition API — reactivity, Props, Emits, and composables — feel far less abstract than they do on the page of a tutorial. Watching a single keystroke travel from an input field, through an emitted event, into a shared reactive object, and out onto the screen and into Local Storage is, in the end, the clearest possible demonstration of what those ideas are actually for.

References

[1] Vue.js Core Team, "Vue 3 Documentation," vuejs.org, 2024.

[2] Vue Router Team, "Vue Router Documentation," router.vuejs.org, 2024.

[3] MDN Web Docs, "Window: localStorage property," developer.mozilla.org, Mozilla.

[4] MDN Web Docs, "Window: print() method," developer.mozilla.org, Mozilla.

[5] MDN Web Docs, "CSS Media Queries: print," developer.mozilla.org, Mozilla.

[6] ECMA International, "ECMAScript 2020 Language Specification — Modules," ecma-international.org.